PROOF OF CONCEPT

CardioTwin system architecture

Synthetic patient and wearable signals move through a risk workflow to clinician and patient apps.

01 DATA INPUTS

Synthetic sources

Synthetic EHR — SYNTHETIC

Seeded demographics, history and baseline

- Demo patient records
- Baseline context for risk checks

Simulated wearable — STREAM

Incoming patient measurements

- Heart rate | blood pressure
- SpO■ | ECG | timestamp

EHR context

vitals JSON

02 CARDIOTWIN BACKEND

FastAPI | Python | in-memory demo state

FastAPI service — API :8000

REST endpoints + WebSocket telemetry

| /health | /docs | patient and alert routes

Risk assessment workflow — CORE

Evaluate latest vitals with patient context

| Cardiac event probability and risk category

| Arrhythmia and blood-pressure checks

Demo runtime state — NO DATABASE

Process memory; resets on restart

Patients | latest vitals | alerts

Prototype data is not persisted.

Alert handling — RULES

Create, list and acknowledge alerts

Risk levels: LOW | MODERATE

HIGH | CRITICAL

INGESTION

POST /api/patients/{id}/vitals

JSON request | demo endpoint

API docs available at /docs

RESULTS RETURNED TO THE CLIENTS

Risk category · Probability · Vitals trends · Alerts

The dashboard and mobile app refresh through API responses and live telemetry.

03 APPLICATIONS

Local demo clients

Clinician dashboard — WEB

Next.js web app | local :3000

- Risk mix and patient charts
- Prediction watchlist + alert queue

Patient mobile app — MOBILE

Expo / React Native

- Patient status and telemetry
- Alerts and profile

CLIENT CONNECTIONS

REST / JSON

WebSocket for live telemetry

API serves both applications

PROTOTYPE NOTES

Built for synthetic-data demonstration and system exploration.

Predictions are demo outputs only; the project is not intended for diagnosis or clinical decision-making.

NO PERSISTENT DATABASE

Runtime data stays in memory and resets on restart.